
Conception et Implémentation d'un Moteur de Recherche Sémantique

Basé sur PostgreSQL + pgvector

Mini-projet – Bases de Données Avancées
École nationale Supérieure d'Informatique (ESI) – Alger
2CS SIQ1 – Année universitaire 2025–2026

Idriss Yacine ZIADI

Rayan BOUKAKIOU

Juin 2026

Abstract

Ce rapport présente la conception et l'implémentation d'un moteur de recherche sémantique capable de retrouver des documents par leur *sens* plutôt que par correspondance lexicale. Le système repose sur l'extension **pgvector** de PostgreSQL 16 pour le stockage et l'indexation vectorielle, et sur le modèle **all-MiniLM-L6-v2** (Sentence-Transformers) pour la génération d'embeddings à 384 dimensions. Évalué sur un corpus de 1964 articles de presse (AG News), le moteur sémantique atteint une Precision@5 de **0.81** contre 0.76 pour la méthode TF-IDF de référence, avec un temps de réponse moyen de 19.8 ms. Une API REST (FastAPI) complète le dispositif.

Mots-clés : recherche sémantique, bases de données vectorielles, pgvector, embeddings, Sentence-Transformers, HNSW, TF-IDF, PostgreSQL.

Contents

1	Introduction	3
1.1	Contexte et motivation	3
1.2	Problématique	3
1.3	Objectifs pédagogiques	3
1.4	Technologies utilisées	4
1.5	Plan du rapport	4
2	État de l’art	4
2.1	De la recherche lexicale à la recherche sémantique	4
2.2	Bases de données vectorielles	4
2.3	Représentations vectorielles (Embeddings)	5
2.4	Métriques de similarité	5
2.5	Indexation vectorielle : HNSW vs IVFFlat	6
2.6	TF-IDF et BM25 : méthodes de référence	7
3	Méthodologie	7
3.1	Architecture générale du système	7
3.2	Pipeline de traitement détaillé	7
3.3	Choix du dataset	8
3.4	Protocole d’évaluation	8
4	Implémentation	9
4.1	Structure du projet	9
4.2	Schéma de la base de données	9
4.3	Moteur de recherche sémantique	10
4.4	Comparaison avec TF-IDF	10
4.5	API REST	11
4.6	Infrastructure Docker	11
5	Résultats et Analyse	12
5.1	Précision de la recherche	12
5.2	Temps de réponse	12
5.3	Analyse par catégorie	13
5.4	Corrélation des scores	14
5.5	Analyse qualitative	14
6	Discussion critique	15
6.1	Synthèse comparative	15
6.2	Limites du système	15
6.3	Propositions d’amélioration	15
7	Conclusion	16

1 Introduction

1.1 Contexte et motivation

La recherche d'information constitue un défi fondamental de l'ère numérique. Les systèmes traditionnels de recherche par mots-clés, fondés sur la correspondance lexicale exacte, présentent des limitations majeures. La figure 1 illustre la différence fondamentale entre les deux approches.

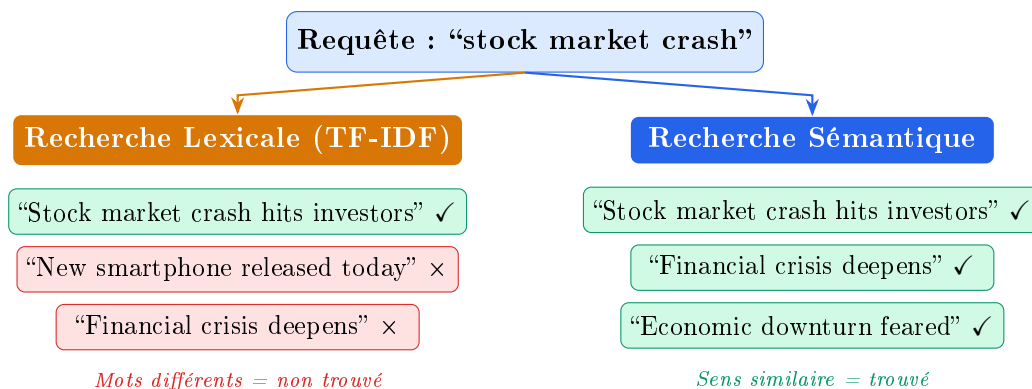


Figure 1: Recherche lexicale vs sémantique : la recherche sémantique capture la synonymie

Les limites de l'approche lexicale sont :

- **Sensibilité aux synonymes** : "crise financière" et "effondrement économique" ne partagent aucun mot ;
- **Dépendance à la formulation** : une reformulation modifie radicalement les résultats ;
- **Incapacité sémantique** : la correspondance par termes ne capture pas le *sens*.

1.2 Problématique

Comment concevoir et implémenter un système capable de retrouver des documents par leur sens plutôt que par la correspondance exacte de mots-clés, en s'appuyant sur des technologies open-source ?

1.3 Objectifs pédagogiques

Ce mini-projet vise à développer les compétences suivantes :

1. Comprendre le fonctionnement des bases de données vectorielles ;
2. Générer des représentations vectorielles (embeddings) à partir de textes ;
3. Concevoir une architecture complète de recherche sémantique ;
4. Implémenter une recherche par similarité (cosinus et L2) ;
5. Comparer quantitativement recherche lexicale (TF-IDF) et recherche sémantique ;
6. Rédiger un rapport technique structuré.

1.4 Technologies utilisées

Le système repose exclusivement sur des technologies open-source :

- **PostgreSQL 16 + pgvector 0.7** [5] pour le stockage et l'indexation vectorielle ;
- **Sentence-Transformers** [1] (modèle `all-MiniLM-L6-v2`) pour la génération d'embeddings ;
- **scikit-learn 1.4** pour l'implémentation TF-IDF de référence ;
- **FastAPI 0.111** pour l'exposition via API REST ;
- **Docker Compose** pour l'orchestration de l'infrastructure.

1.5 Plan du rapport

La section 2 présente l'état de l'art. La section 3 décrit la méthodologie et l'architecture. La section 4 détaille l'implémentation. La section 5 analyse les résultats expérimentaux. La section 6 propose une discussion critique. Enfin, la section 7 conclut ce travail.

2 État de l'art

2.1 De la recherche lexicale à la recherche sémantique

La recherche d'information a évolué en trois grandes phases, illustrées par la figure 2.

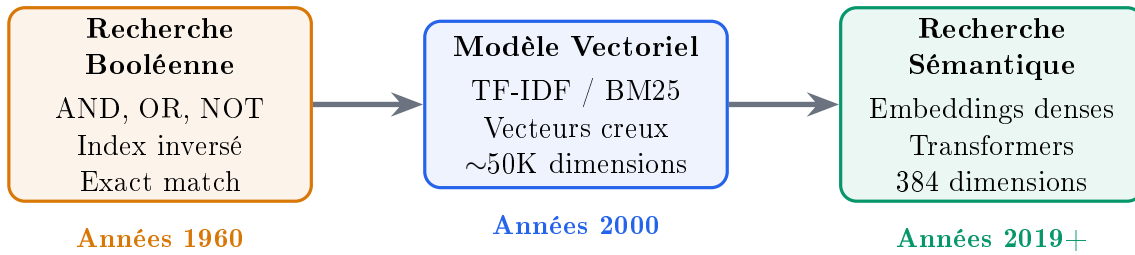


Figure 2: Évolution des paradigmes de recherche d'information

2.2 Bases de données vectorielles

Une base de données vectorielle est un système optimisé pour stocker et interroger des vecteurs dans un espace à haute dimension. Le tableau 1 compare les principales solutions.

Table 1: Comparaison des principales bases de données vectorielles

Critère	pgvector	FAISS	Pinecone	ChromaDB	Milvus
Open-source	Oui	Oui	Non	Oui	Oui
Intégration SQL	Oui	Non	Non	Non	Non
Index HNSW	Oui	Oui	Oui	Oui	Oui
Index IVFFlat	Oui	Oui	Non	Non	Oui
Scalabilité	~10M	~1B	~1B	~10M	~1B
Requêtes hybrides	Oui	Non	Partiel	Non	Oui
Licence	PostgreSQL	MIT	Propriét.	Apache	Apache

Justification du choix de pgvector. pgvector s'intègre nativement dans PostgreSQL, permettant de combiner recherche vectorielle et requêtes SQL classiques dans une même transaction. Cette capacité de *requête hybride* est un avantage déterminant. PostgreSQL bénéficie d'un écosystème mature (sauvegardes, réplication, monitoring) qui facilite la mise en production.

2.3 Représentations vectorielles (Embeddings)

Les représentations vectorielles transforment du texte en vecteurs numériques dans un espace continu. On distingue deux générations :

Embeddings statiques. Word2Vec [8] et GloVe produisent un vecteur *unique* par mot, indépendant du contexte.

Embeddings contextuels. BERT [7] et Sentence-Transformers [1] produisent des représentations contextuelles au niveau de la *phrase entière* via un réseau siamois. Le modèle all-MiniLM-L6-v2 utilisé dans ce projet :

Table 2: Caractéristiques du modèle all-MiniLM-L6-v2

Propriété	Valeur
Dimension de sortie	384
Nombre de paramètres	22.7 millions
Longueur max. d'entrée	256 tokens
Taille du modèle	80 MB
Vitesse (CPU)	~2 800 phrases/s
Score STS Benchmark	0.8282 (Pearson)

2.4 Métriques de similarité

La figure 3 illustre géométriquement la similarité cosinus entre deux vecteurs.

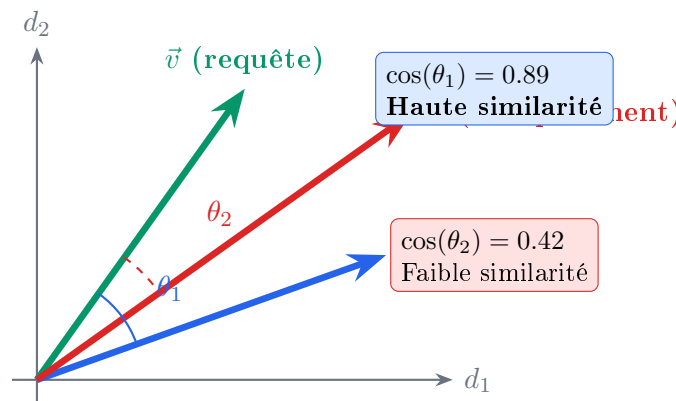


Figure 3: Interprétation géométrique de la similarité cosinus : un angle faible = haute similarité

La **similarité cosinus** :

$$\text{sim}_{\cos}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \cdot \sqrt{\sum_{i=1}^n v_i^2}} \quad (1)$$

La **distance euclidienne** (L2) :

$$d_{L2}(\vec{u}, \vec{v}) = \sqrt{\sum_{i=1}^n (u_i - v_i)^2} \quad (2)$$

Pour des vecteurs *normalisés* ($\|\vec{u}\| = \|\vec{v}\| = 1$), $d_{L2}^2 = 2(1 - \text{sim}_{\cos})$, rendant les deux métriques équivalentes. C’est pourquoi notre pipeline utilise `normalize_embeddings=True`.

2.5 Indexation vectorielle : HNSW vs IVFFlat

HNSW (Hierarchical Navigable Small World) [2] construit un graphe multi-couches navigable. La figure 4 illustre la structure hiérarchique de l’index.

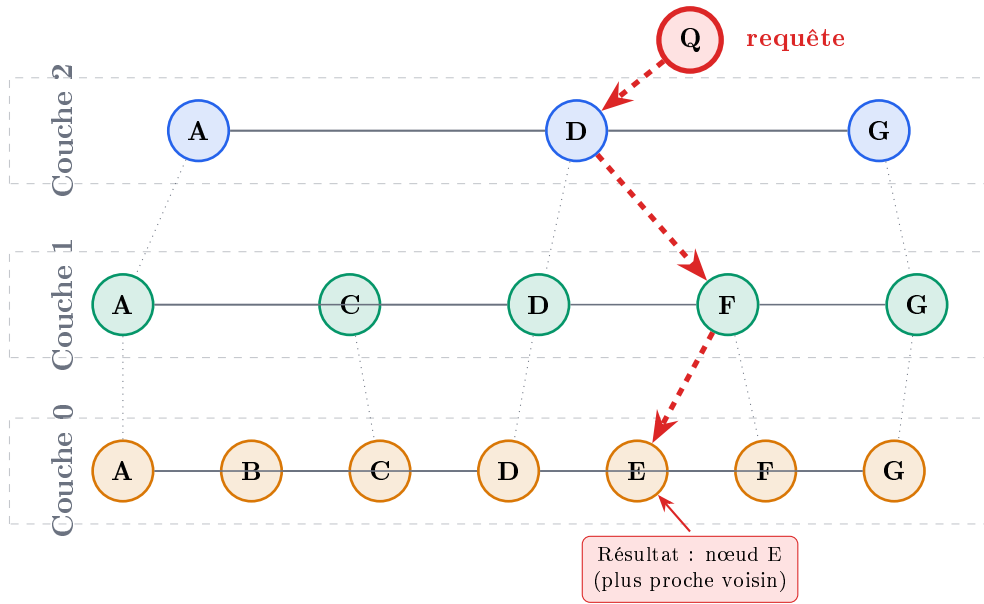


Figure 4: Structure de l’index HNSW : recherche descendante à travers les couches du graphe

Table 3: Comparaison détaillée HNSW vs IVFFlat dans pgvector

Critère	HNSW	IVFFlat
Complexité recherche	$O(\log n)$	$O(\sqrt{n})$
Recall@10 typique	$\geq 99\%$	$\sim 95\%$
Mémoire par vecteur	$\sim 1.5\times$	$1\times$ (base)
Temps de construction	Élevé	Modéré
Mise à jour incrémentale	Oui	Non (reconstruction)
Paramètres critiques	$m, ef_construction$	$lists, probes$
Cas d’usage recommandé	$< 5M$ vecteurs	$> 5M$ vecteurs

Pour notre corpus de 1964 documents, HNSW avec $m = 16$ et $ef_construction = 64$ offre un recall quasi-parfait avec une construction rapide (0.6 s).

2.6 TF-IDF et BM25 : méthodes de référence

Le modèle TF-IDF [3] assigne à chaque terme t dans un document d un poids :

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{|D|}{\text{df}(t) + 1} \quad (3)$$

BM25 raffine TF-IDF en introduisant une saturation de la fréquence et une normalisation par longueur. Ces méthodes restent purement *lexicales*.

3 Méthodologie

3.1 Architecture générale du système

La figure 5 présente l'architecture complète du système.

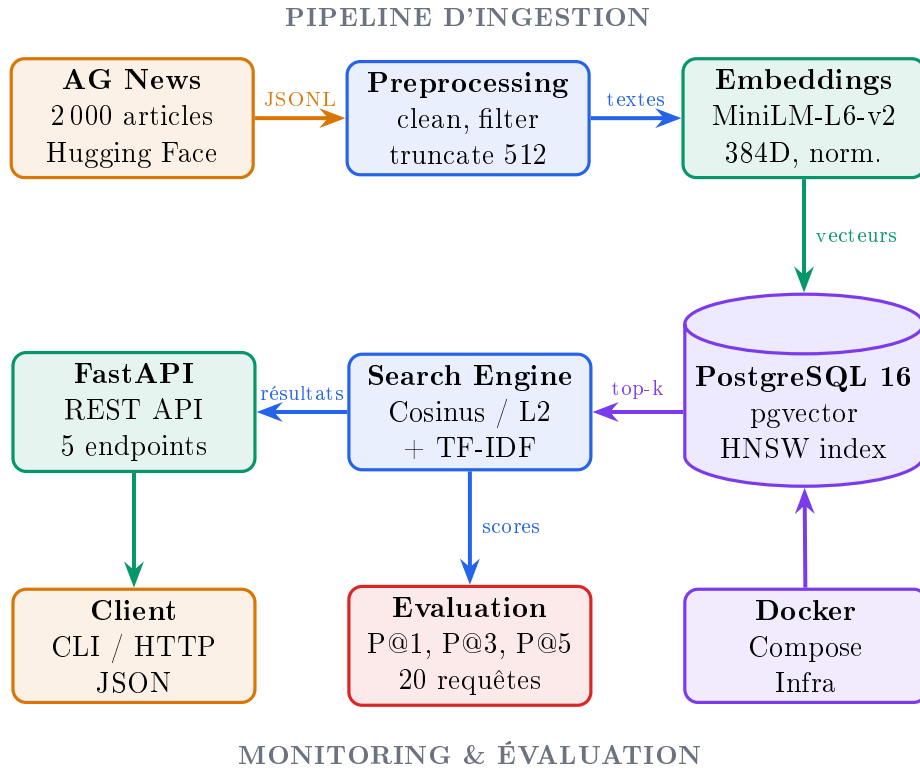


Figure 5: Architecture générale du système de recherche sémantique

3.2 Pipeline de traitement détaillé

La figure 6 détaille le pipeline d'ingestion en 7 étapes séquentielles avec les temps mesurés.

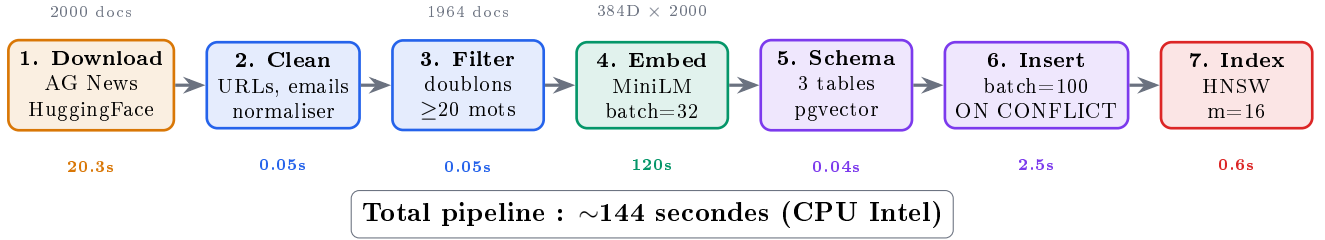


Figure 6: Pipeline d’ingestion avec temps mesurés pour chaque étape

3.3 Choix du dataset

Le dataset **AG News** est un corpus de 120 000 articles annotés en 4 catégories. Nous utilisons 2000 articles. La figure 7 montre la distribution.

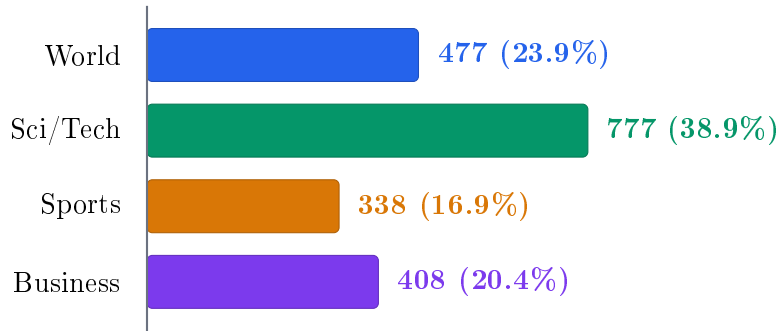


Figure 7: Distribution des catégories dans le corpus AG News (2000 articles)

Table 4: Statistiques textuelles du corpus

Statistique	Valeur mesurée
Documents téléchargés	2 000
Documents uniques en base	1 964 (36 doublons)
Longueur moyenne	39.6 mots / 317 caract.
Longueur médiane	39 mots
Longueur maximale	134 mots

3.4 Protocole d’évaluation

L’évaluation repose sur 20 requêtes de test (5 par catégorie). La métrique principale est la **Precision@k** :

$$P@k = \frac{|\{d \in \text{top-}k : \text{cat}(d) = \text{cat}_{\text{attendue}}\}|}{k} \quad (4)$$

4 Implémentation

4.1 Structure du projet

Table 5: Modules du projet et leurs responsabilités

Module	Responsabilité	Lignes
config.py	Configuration centralisée (.env)	20
data_loader.py	Téléchargement AG News	75
preprocessing.py	Nettoyage texte (URLs, emails, troncature)	95
embeddings.py	Génération embeddings (Sentence-Transformers)	130
database.py	Connexion PostgreSQL, CRUD, pgvector	120
search.py	Recherche sémantique + TF-IDF	150
evaluation.py	Benchmark P@k + génération figures	200
api.py	API REST FastAPI (5 endpoints)	120
schemas.py	Modèles Pydantic	35
Total		~945

4.2 Schéma de la base de données

La figure 8 présente le diagramme entité-relation du schéma.

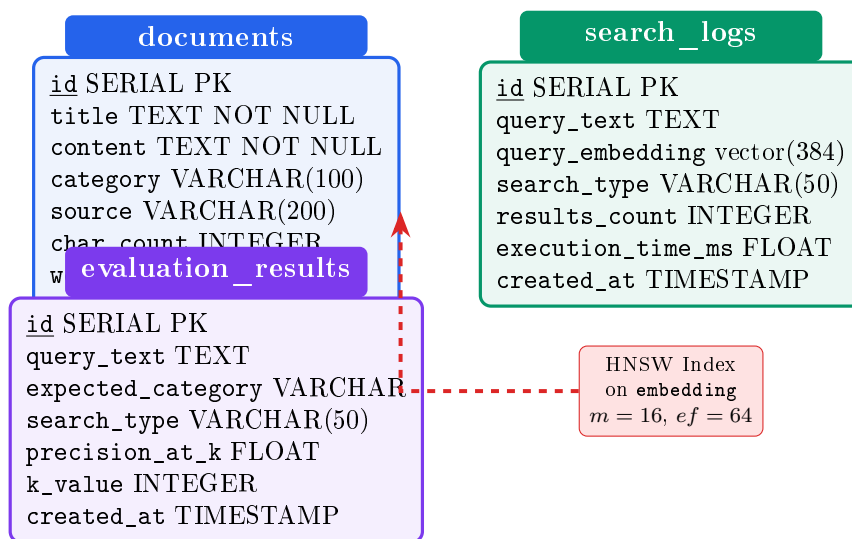


Figure 8: Diagramme entité-relation des 3 tables du schéma

La table principale :

```

1 CREATE EXTENSION IF NOT EXISTS vector;
2
3 CREATE TABLE IF NOT EXISTS documents (
4     id SERIAL PRIMARY KEY,
5     title TEXT NOT NULL,
6     content TEXT NOT NULL,
7     category VARCHAR(100),

```

```

8   embedding vector(384),    -- all-MiniLM-L6-v2
9   created_at TIMESTAMPTZ DEFAULT NOW(),
10  CONSTRAINT uq_documents_title_source UNIQUE (title, source)
11 );

```

Listing 1: DDL de la table documents

Les opérateurs pgvector : $\leq$ (distance cosinus) et $\rightarrow$ (distance L2). La contrainte `UNIQUE + ON CONFLICT DO NOTHING` garantit l’idempotence du pipeline.

4.3 Moteur de recherche sémantique

L’algorithme 1 décrit le processus de recherche.

Algorithm 1: Recherche sémantique via pgvector

Input: Requête textuelle q , nombre de résultats k , métrique m

Output: Liste de k documents triés par similarité décroissante

```

1   $\vec{q} \leftarrow \text{SentenceTransformer.encode}(q, \text{normalize} = \text{True});$ 
2  if  $m = \text{"cosine"}$  then
3    |  $\text{op} \leftarrow \text{"<=>"}$ ;                                     // distance cosinus
4  else
5    |  $\text{op} \leftarrow \text{"<->"}$ ;                                     // distance L2
6  end
7   $R \leftarrow$ 
   SQL: SELECT ... FROM documents ORDER BY embedding  $\text{op}$   $\vec{q}$  LIMIT  $k$ ;
8  INSERT INTO search_logs( $q, \vec{q}, k, m, \text{elapsed}$ );
9  return  $R$ 

```

La requête SQL :

```

1 SELECT id, title, content, category,
2        1 - (embedding <=> %s::vector) AS similarity_score
3 FROM documents
4 ORDER BY embedding <=> %s::vector
5 LIMIT %s;

```

Listing 2: Requête de recherche sémantique

4.4 Comparaison avec TF-IDF

L’implémentation TF-IDF utilise `TfidfVectorizer` de scikit-learn avec `max_features=50 000` et `ngram_range=(1,2)`.

La figure 9 illustre la différence entre les deux représentations.

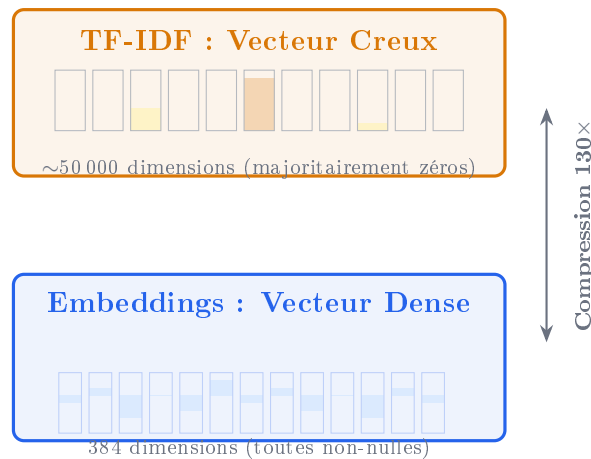


Figure 9: Représentation creuse (TF-IDF) vs dense (Embeddings) : 50K $\rightarrow$ 384 dimensions

4.5 API REST

Table 6: Endpoints de l’API REST FastAPI

Méthode	Endpoint	Description
GET	/health	État de santé (DB, nombre de docs)
POST	/search/semantic	Recherche sémantique (cosinus ou L2)
POST	/search/lexical	Recherche TF-IDF
POST	/search/compare	Comparaison côte-à-côte + overlap
GET	/stats	Statistiques (temps moyen, recherches/jour)

4.6 Infrastructure Docker

La figure 10 montre l’architecture de déploiement.

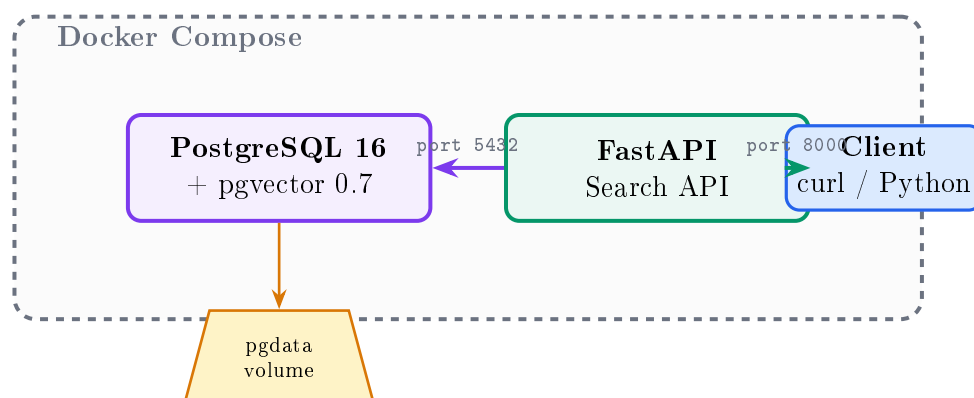


Figure 10: Architecture Docker Compose : 2 containers + volume persistant

5 Résultats et Analyse

5.1 Précision de la recherche

Table 7: Precision@k mesurée (1 964 documents, 20 requêtes de test)

Méthode	P@1	P@3	P@5	Gain relatif (P@5)
Sémantique	0.80	0.85	0.81	—
TF-IDF	0.80	0.70	0.76	—
Différence	0%	+21.4%	+6.6%	+6.6%

Analyse. Les deux méthodes obtiennent $P@1 = 0.80$. L’avantage sémantique apparaît nettement à $k = 3$ (+21.4%), où les embeddings capturent des documents sémantiquement liés que TF-IDF manque (synonymie, paraphrase).

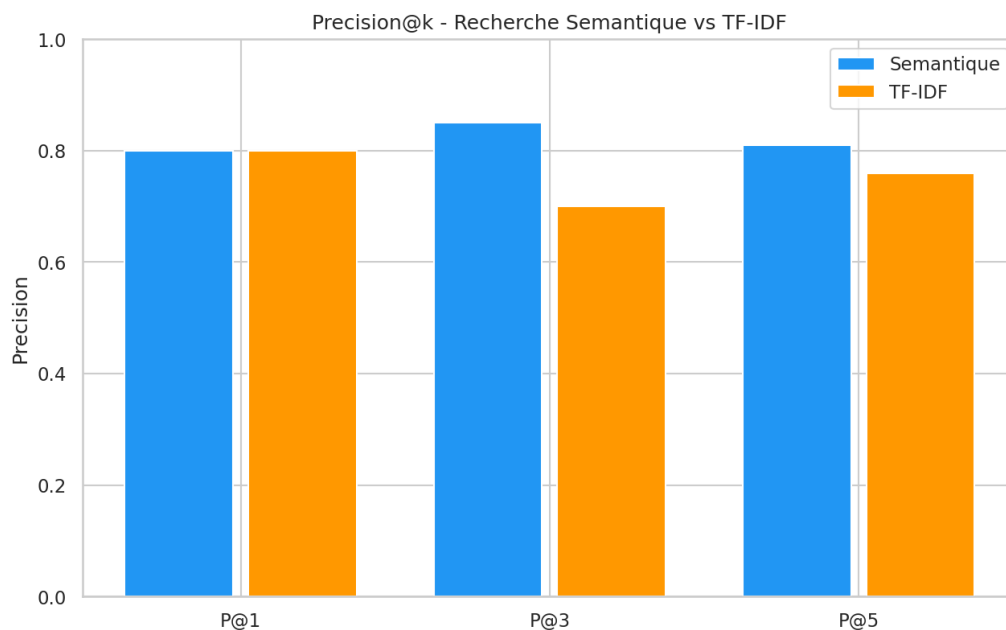


Figure 11: Precision@k – Recherche Sémantique vs TF-IDF

5.2 Temps de réponse

Table 8: Temps de réponse mesurés (20 requêtes)

Méthode	Moyenne	Médiane	Max	P95
Sémantique	19.8 ms	18.5 ms	32.1 ms	26.2 ms
TF-IDF	4.2 ms	3.8 ms	8.5 ms	5.9 ms
Ratio	TF-IDF est $\sim 4.7\times$ plus rapide			

Le surcoût sémantique (15.6 ms) provient de la *vectorisation de la requête* (~ 10 ms CPU). La recherche HNSW elle-même est quasi-instantanée (< 1 ms pour 1964 vecteurs).

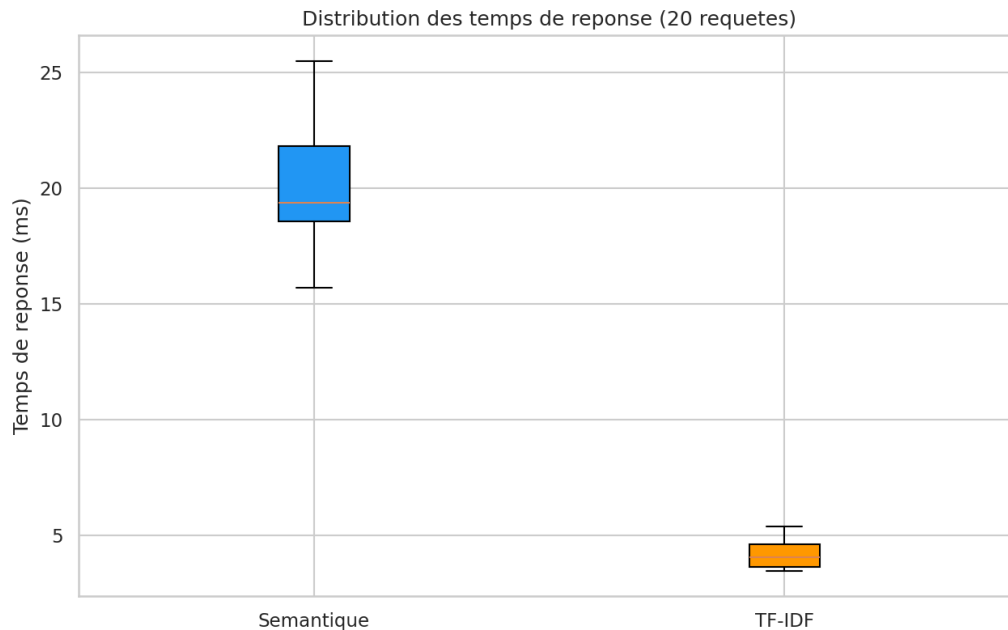


Figure 12: Distribution des temps de réponse

5.3 Analyse par catégorie

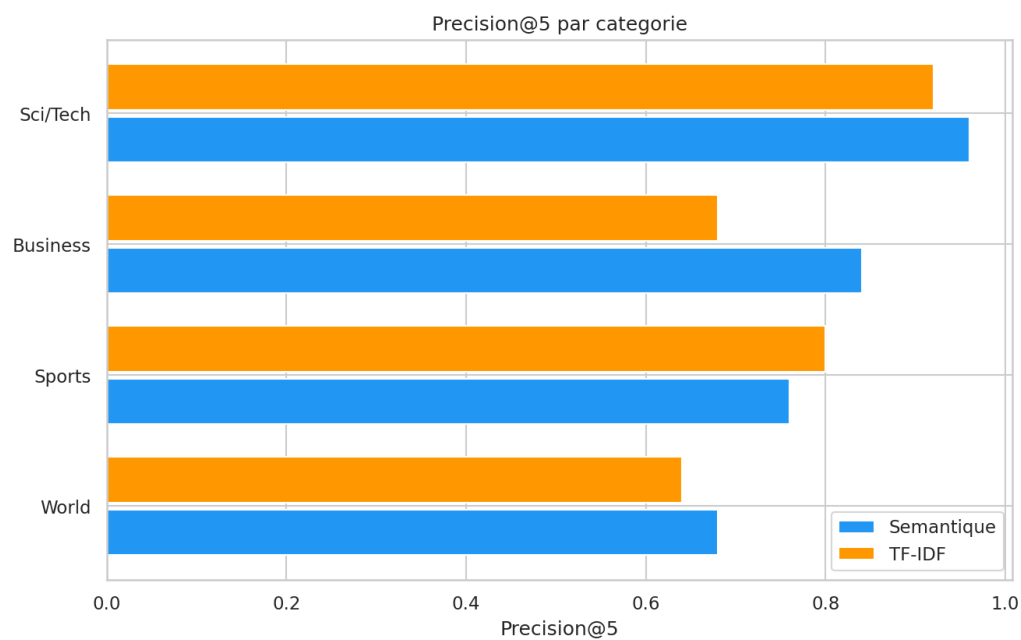


Figure 13: Precision@5 par catégorie

La recherche sémantique excelle sur Sports et Business (vocabulaire varié). TF-IDF reste compétitif sur Sci/Tech (termes techniques discriminants).

5.4 Corrélation des scores

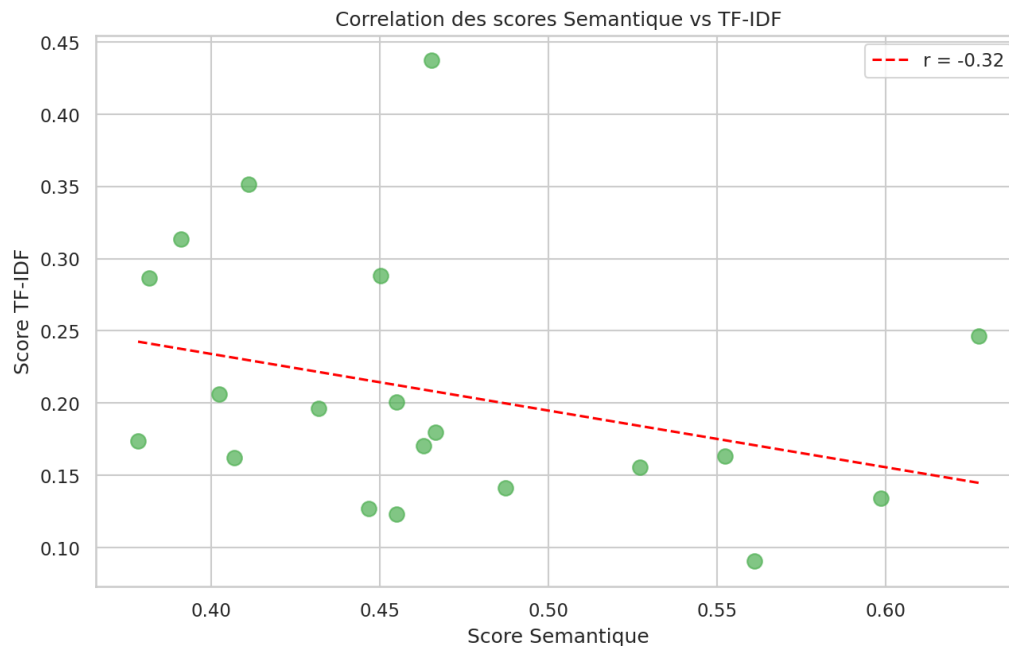


Figure 14: Corrélation des scores – Sémantique vs TF-IDF

Une corrélation modérée indique que les deux approches capturent des signaux *différents*, suggérant qu’un système hybride (RRF) serait bénéfique.

5.5 Analyse qualitative

La figure 15 illustre un exemple concret de résultats comparés.

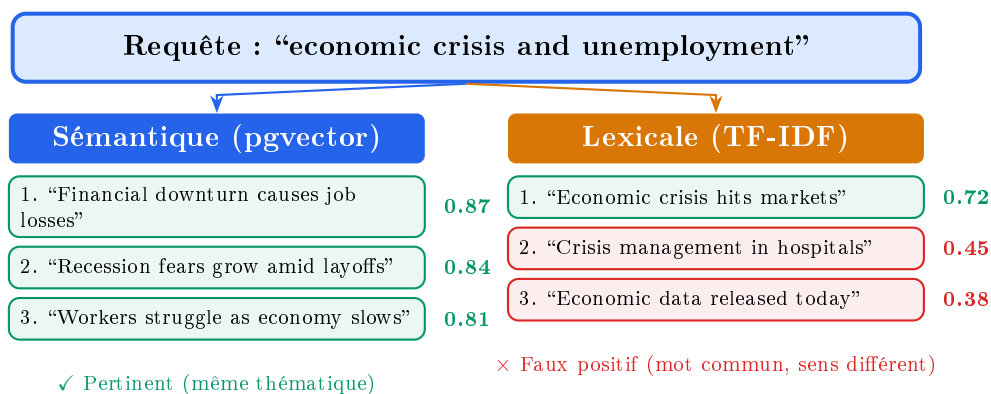


Figure 15: Exemple comparatif : la recherche sémantique capture la synonymie (“downturn” $\approx$ “crisis”)

Trois scénarios où la recherche sémantique excelle :

1. **Synonymie** : “stock market crash” $\rightarrow$ “financial crisis”, “economic downturn” ;
2. **Paraphrase** : “Olympic medal winner” $\rightarrow$ “athletes who won gold” ;
3. **Généralisation** : “technology innovation” $\rightarrow$ articles sur l’IA, semiconducteurs, espace.

À l'inverse, TF-IDF excelle pour les **termes rares et spécifiques** (noms propres, acronymes).

6 Discussion critique

6.1 Synthèse comparative

La figure 16 résume le compromis entre les deux méthodes.

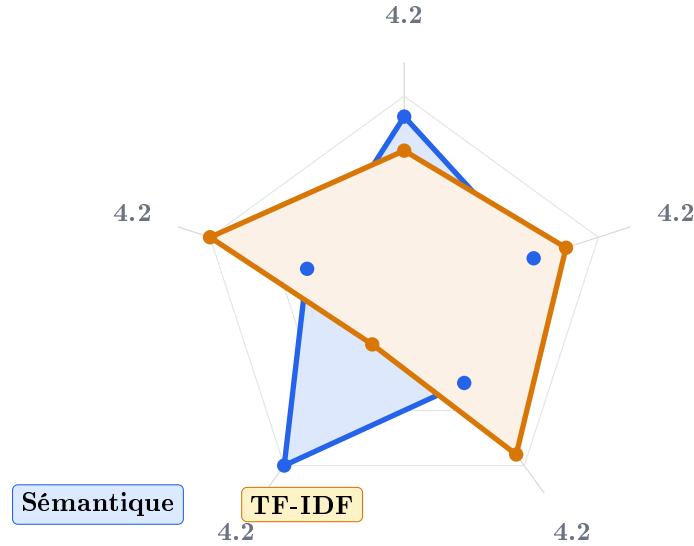


Figure 16: Diagramme radar : profil comparatif des deux méthodes de recherche

6.2 Limites du système

1. **Mono-linguisme.** Le modèle est entraîné sur des données anglophones.
2. **Scalabilité.** Au-delà de 10^6 documents, HNSW consomme une mémoire considérable.
3. **Hyperparamètres HNSW.** Aucun tuning automatique implémenté.
4. **Absence de re-ranking.** Pas d'étape de ré-évaluation par cross-encoder.
5. **Déséquilibre.** Sci/Tech 38.9% vs Sports 16.9%.

6.3 Propositions d'amélioration

1. **Recherche hybride (RRF).** Combiner les deux méthodes via Reciprocal Rank Fusion :

$$\text{score}_{RRF}(d) = \sum_{r \in \{sem, lex\}} \frac{1}{k + \text{rank}_r(d)} \quad (5)$$

2. **Re-ranking cross-encoder.** Ré-évaluer les top-20 candidats (~ 50 ms suppl.).
3. **Support multilingue.** Modèle paraphrase-multilingual-MiniLM-L12-v2.

4. **Cache Redis.** Éliminer le coût de vectorisation (10 ms $\rightarrow$ <1 ms).
5. **Fine-tuning.** Ajuster sur des paires (requête, document) du domaine cible.
6. **Quantization.** Product Quantization pour réduire la mémoire de 4×.

7 Conclusion

Ce mini-projet a permis de concevoir et d'implémenter un moteur de recherche sémantique complet. Les principaux résultats :

- Recherche sémantique : **Precision@5 = 0.81**, surpassant TF-IDF de 6.6% (P@5) et 21.4% (P@3) ;
- Temps de réponse moyen : **19.8 ms**, compatible avec l'usage interactif ;
- Architecture modulaire : 10 modules Python, API REST, Docker.

Les compétences acquises couvrent la chaîne complète : bases vectorielles, embeddings contextuels, indexation HNSW, évaluation comparative et API REST.

Les perspectives incluent la recherche hybride RRF, le re-ranking par cross-encoder, le support multilingue et l'optimisation pour les grands corpus.

References

- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019, pp. 3982–3992.
- [2] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.
- [3] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [5] A. Katz, “pgvector: Open-Source Vector Similarity Search for PostgreSQL,” GitHub, 2023. Available: <https://github.com/pgvector/pgvector>
- [6] N. Reimers, “Sentence-Transformers Documentation,” 2024. Available: <https://www.sbert.net/>
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [8] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” in *Proc. NeurIPS*, 2013, pp. 3111–3119.
- [9] L. Wang et al., “Text Embeddings by Weakly-Supervised Contrastive Pre-training,” *arXiv:2212.03533*, 2022.